

e are all responsible for quality.

That being said, we acknowledge that Software Engineers in Tests (SETs) have the most knowledge about writing E2E tests. A benefit of splitting the E2E specs into a separate MR is that it can be assigned to someone other than the DRI of the feature, which lets a more appropriate person write the spec. This can be the SET, backend or frontend, depending on the circumstances.

Given the Ruby expertise required to write E2E tests, we should have SETs and backend engineers be the primary team members to write them. Frontend engineer may volunteer to write them with the support of a SET if they feel confident, but it is not expected of them.

Whenever possible, backend engineers should help write the E2E tests of a new feature or bug fix.

They should feel comfortable pinging the SET on their team as needed (to review the MR, for example).

This helps alleviate the SET's workload, so that they are not entirely responsible for E2E tests on their own.

However when they are not comfortable writing the required E2E tests, then the plan should be for

the SET to lead the effort. The rationale being that SETs have the most context and work daily with specs,

so they will be able to write them much faster. More importantly, they can write much better specs.

DRI engineers should proactively help the SET understand the required tests.

Accounting for capacity

In the instance where the SET has too many E2E tests to write, then they should check with backend engineers of the team if they could lead the effort on some of them. Because testing is part of the required work, we should account for E2E tests when assigning issues to a milestone.

Bug fixes that require additional test coverage

When we fix a bug, we ideally want to write a test that would catch and prevent this bug from happening again in the future.

However, if the spec we need to write (unit, integration, or E2E test), is part of a code change that needs to be merged as

soon as possible (e.g. requires a time sensitive resolution), it is preferable to merge the fix first,

and create an issue to write the spec afterwards, so that it is not blocking merging of the MR.

This example should be considered the exception, not the rule.

Creating follow-up issues for specs

When creating follow-up issues for tests, we have to ensure that they are not going to sit on a

pile in the backlog.

Tests are essential and should be prioritized as such.

When creating a follow-up issue for required tests:

- Make sure to assign the issue to the following milestone and make room for it to be worked on.
- Add all the context necessary to test the feature in the description, for example: Where it is, how it works, how to configure it, etc.
- Assign a DRI immediately and cc the product manager so that they are aware of the issue and required work.

Bugs

As a group, we strive to meet the Severity Service Level Objective of bugs. We regularly review all bugs and prioritize issues with a ~missed-SLO label and those approaching SLO (Service Level Objective) through our weekly Triage Report. One of the group's goals is to reduce the median age of open S2 bugs which is being tracked by the Quality department as a KPI. To do this we will triage aged bugs each milestone closing what we can, reducing severity for bugs mis-labeled, asking for more details for issues that cannot be reproduced and prioritizing those that can be reproduced focusing on bugs in the identified JTBD.

Features

When we build new features, in line with our Iteration value, we aim for an MVC that covers the core aspect of the feature.

Then we create issues with ~feature::enhancement label to iterate on the remaining functionality. The more a new feature is used, the more missing functionality eventually can be perceived as bugs by the users.

We use severity::1, severity::2, severity::3 and severity::4 labels with ~feature::enhancement to classify the impact of those issues to prevent them from becoming ~type::bug.

We use the list below as a guideline to grade the impact.

Label	Definition	Description
-----	-----	-----
severity::1	Blocking or Critical	The enhancement is critical to the core of the feature. The feature is currently incomplete or extra action is required.
severity::2	High	Some important aspects of the feature are missing but the feature delivers value. However the missing behavior can make the feature appear buggy.
severity::3	Medium	The feature works ok but the enhancement is required under less common scenarios.
severity::4	Low	The feature works fine and the improvement aims to improve customer satisfaction.

Technical debt

We track our technical debt using the following Pipeline Execution Technical Debt issue board, where we track issues in the planning phase.

This board has 2 main sections:

1. Issue readiness

- In workflow::planning breakdown we find issues that are currently being refined.
- In workflow::ready for development we find issues that clearly defined and a weight has been assigned.

1. Impact breakdown. We use severity::1, severity::2, severity::3 and severity::4 labels to classify the impact of the specific tech debt item.

We use the list below as a guideline to grade the impact.

- severity::1 Blocking or Critical
 - blocking debt that prevents any further changes in the area
 - we stumble on the same complex code every single time and it causes serious slow down on development
 - the problem exists in a core part of our domain that represents a dependency for a large number of features
 - the problem is wide spread throughout several domains
 - the problem is related to a number of reported severity::1 bugs
- severity::2 High
 - blocks many changes in the area or makes them difficult to work around it
 - the problem exists in an area that changes frequently, causing it or workarounds to spread easily
 - the problem is related to an important feature (e.g multi-project pipelines)
 - the problem is very common in a specific domain and leaks into other domains too
 - the problem is related to a number of reported severity::2 bugs
- severity::3 Medium
 - the problem exists in a feature that has a supporting role in our domain
 - the problem exists in an area that does not change frequently
 - the problem is very common in a specific domain but limited to its domain boundaries
 - workarounds slow down development
 - the problem is related to a number of reported severity::3 bugs
- severity::4 Low
 - the problem is very isolated and has low or no impact to development
 - the area affected does not change frequently
 - the problem is related to a number of reported severity::4 bugs

1. Issue scheduling. The Engineering Manager(s) will review the PE Technical Debt issue board and recommend issues to schedule for upcoming milestones by adding them to the planning issue for a milestone or communicating with Product a need to schedule the issue in an upcoming milestone. The team strives to have 20% of a sprint's capacity filled with Tech Debt issues.

Note that multiple factors can exist at once. In that case use your judgment to either bump the impact score or lower it. For example:

- The problem exists in a feature that has a supporting role in our domain but it's related to a number of severity::2 bugs.

Then choose severity::2.

- The problem is related to an important feature but the workaround is acceptable as the code around it does not change frequently. Then choose severity::3.

Risk Mapping

To better understand the risk environment and each risk's causes and consequences, the Pipeline Execution team uses the Risk Map as our risk management tool to prioritise mitigation strategies and increase Quality.

Retrospectives

```
{{% include "includes/engineering/verify-retrospectives.md" %}}
```

Team Communication

```
{{% engineering/verify-team-communication slack-channel="gpipeline-execution" slack-  
url="https://gitlab.slack.com/archives/CPCJ8CCCX" group-label="group:pipeline execution" group-  
issues-url="https://gitlab.com/groups/gitlab-  
org/-/issues?labelname%5B%5D=group%3A%3Apipeline+execution&scope=all", gitlabsubgroup:  
'pipeline-execution-group" %}}
```

Developer Onboarding

Refer to the Developer Onboarding in Verify section.

```
## SECTION: engineering/devops/ops/verify/pipeline-execution/jtbd.md
```

Overview

The goal of this page is to document, share and iterate on the Jobs to be Done (JTBD) and their corresponding job statements for the Pipeline Execution group. Using JTBD framework we intend to arrive at the more specific problems to be solved in relation to Continuous Integration workflows.

Goals

Utilize JTBD and job statements to:

- Understand our users' underlying motivations
- Validate identified problem areas within the stage-group
- Create a common language across teams for better collaboration when working on improving the experience for GitLab CI
- Create a transparent view for our stakeholders into the current and future state of the product.

JTBD

```
{{% product/jtbd "Pipeline Execution" %}}
```

```
## SECTION: engineering/devops/ops/verify/pipeline-execution/risk-map.md
```

Overview

The goal of this page is to create, share and iterate on the Risk Map for the Pipeline Execution group.

Goals

Utilise the Risk Map as a tool to:

- Understand the risks the team faces
- Increase transparency on mitigation plans
- Effectively allocate limited resources
- Collaborate strategically in improving Quality

General Risk Map

Map key

- Impact - what happens if the risk is not mitigated or eliminated
- Impact level - Rate 1 (LOW) to 5 (HIGH)
- Probability - Rate 1 (LOW) to 5 (HIGH)
- Priority - Impact x Probability. Address highest score first.
- Mitigation - what could be done to lower the impact or probability

Risk Area	Risk Description	Impact	Impact level (1 LOW to 5 HIGH)	Probability (1 LOW to 5 HIGH)	Priority	Mitigation
---	---	---	---	---	---	---
Team/Capacity	We have 6 BE engineers and 2 FE engineers on Pipeline Execution and have a large (and growing) backlog	Burn out, missed SLO/SLA, lowers team productivity	5	3	15	Make BE headcount more available
Team/Capacity	Unpredictable throughput	Low Say/Do, Missed SLO/SLA	1	1	1	
Team/Capacity	We no longer have a stable counterpart for UX	Risk to usability and increase SUS bugs. Potential burn out for EM/PM who take over the responsibilities.	5	3	12	Consider scaling other counterparts if the size of the engineering team grows
Team/Capacity	We have a shared (30%) stable counterpart for SET	Escape regression bugs	4	4	16	
Team/Escalations	Escalations like Rapid Actions, Engineering Allocations are disrupting the ability to focus on team priorities	Burn out, low level of autonomy, lowers team productivity	5	4	20	Find ways to proactively mitigate urgent issues with gitlab.com, work on GraphQL to unblock FE, find a dedicated SRE for CI
Product/Backlog	Bug and Technical Debt backlog has been accruing over the years	missed SLO/SLA, prioritization is harder	5	3	15	Revisit ownership of domains to better share the gaps
Infrastructure availability	Pipelines get stuck due to stuck sidekiq shard	Mass failure in E2E test suites and/or customer usage impacted	4	3	12	
Quality/Testability	Hard to replicate production traffic to account for performance testing		4	4	16	
Quality/Test coverage	This is a mature product, there are many features and feature sets have yet to have test coverages (historical test gaps)	Escape regression bugs	4	4	16	

| Product/Cost | CI pipeline inefficiencies | CI Minute usage that could potentially be avoided
| 5 | 5 | 25 | Develop features to optimize pipeline runtime |
| Feature/Performance | Unperformant database queries | Adding load to gitlab.com database,
slow page and feature load times | 3 | 3 | 9 | Recent rapid actions has helped, and there's
continual effort to address this to ensure we don't regress |
| Team/Efficiency | Migrating more REST to GraphQL to help unblock FE | FE productivity and
delivery | 5 | 3 | 15 | |
| Feature/Dependencies | Depends on runner response and processing time
-
<https://gitlab.com/gitlab-org/gitlab/-/issues/326113>
- <https://gitlab.com/gitlab-com/gl-infra/production/-/issues/3631> | If runners fail to process, jobs are not executed, pipeline
is stuck | 5 | 3 | 15 | |
| Infrastructure availability | CI/CD Data model scaling | CI/CD Data model scaling | 5 | 2 |
10 | Actively being worked on in CI/CD Data Model Blueprint MR |

SECTION: engineering/devops/ops/verify/pipeline-execution/project-plans/_index.md

SECTION: engineering/devops/ops/verify/pipeline-execution/project-plans/ci-data-partitioning.md

CI Data Partitioning - Weekly Project Plan

Milestones Goals

- 16.6: cibuilds and cibuildsmetadata tables partitioned
- 16.7: cibuilds and cibuildsmetadata second partitions in use
- 16.9: cipipelinevariables table partitioned
- 17.0: cijobartifacts, cistages tables partitioned
- 17.2: cipipelines table partitioned
- 17.3: cipipelines, cijobartifacts, cistages, and cipipelinevariables second partitions in use
- 17.6: (very rough estimate) start rebalancing for partitioned tables
- 17.9: (very rough estimate) complete rebalancing for partitioned tables

-

Related Epic

<details markdown="1">

<summary markdown="span">Archive</summary>

Week of August 14, 2023

Team Capacity

- 1 BE

Goals

cibuilds

- x] Merge [MR to use routing table
- x] Merge [MR fo ensure id uniqueness across partitions
- x] Confirm execution of [MR on .com or retry in case of failure(autovacuum conflict is still an option)

Week of August 21, 2023

Team Capacity

- 1 BE

Goals

cibuilds

- [~] Fix caching issue blocking enabling use of routing table
- [~] Reintroduce migration for self-managed for ensuring uniqueness of ids across partitions

Week of August 28, 2023

Team Capacity

- 0 BE (Note: Marius focused on customer escalation)

Goals

Week of September 4, 2023

Team Capacity

- 0 BE (Note: Marius focused on customer escalation)

Goals

cibuilds

- ~] [Analyze partitioned parent tables on a schedule
- ~] [Manually analyze pcibuilds and pcibuildsmetadata
- ~] Deploy the [table name switch to canary and confirm there are no more timeouts

Week of September 11, 2023

Team Capacity

- 1 BE

Goals

cibuilds

- [~] Fix caching issue blocking enabling use of routing table
- ~] Enable query analyzers: [Issue

Milestone 16.5 (September 18, 2023 - October 16, 2023)

Week of September 18, 2023

Team Capacity

- 1 BE

Goals

cibuilds

- x] [Manually analyze pcibuilds and pcibuildsmetadata
- x] Deploy the [table name switch to canary and confirm there are no more timeouts

Week of September 25, 2023

Team Capacity

- 1 BE

Goals

cibuilds

- ~] [Analyze partitioned parent tables on a schedule
- ~] Enable query analyzers: [Issue

Week of October 2, 2023

Team Capacity

- 1 BE

Goals

This week we aim to complete the tooling adaptations to allow query analyzers for partitioned parent tables to run on a schedule. This will allow us to ensure the autovacuum process runs on partitioned tables.

cibuilds

- ~] [Analyze partitioned parent tables on a schedule

Week of October 9, 2023

Team Capacity

- 1 BE

Goals

This week we aim to switch cibuilds to use the routing table rather than directly using the partition. We will also enable the query analyzers for the cibuilds and cibuildsmetadata table to ensure their performance is not being significantly negatively impacted. We will continue to work on MRs to update filters in queries to use the partitionid.

Status Update

We are still coordinating with infrastructure to switch the table name. Hopefully next week this can be completed.

Task List

cibuilds

- [~] Switch the table name on the entire fleet
- ~] Enable query analyzers for cibuilds: [Issue (blocked by table name change)
- [x] Enable query analyzers for cibuildsmetadata.
- [x] Start adding partitionid filters to necessary related queries

Week of October 16, 2023

Team Capacity

- 1 BE

Goals

This week we will work on the items that were pushed forward from last week. We aim to switch cibuilds to use the routing table rather than directly using the partition. We will also enable the query analyzers for the cibuilds and cibuildsmetadata table to ensure their performance is not being significantly negatively impacted. We will continue to work on MRs to update filters in queries to use the partitionid.

Task List

cibuilds

- [x] Switch the table name on the entire fleet

- x] [Analyze partitioned parent tables on a schedule
- ~] Enable query analyzers for cibuilds: [Issue (blocked by table name change)

Status Update

The table name got switched on production, so we are able to move forward with the remainder of the partitioning tasks for cibuilds now.

Milestone 16.6 (October 17, 2023 - November 10, 2023)

Team Capacity

- 1 BE

Milestone Goals

This milestone we plan to finish updating the queries to filter on partitionid for cibuilds and cibuildsmetadata.

We will add the migration for self-managed that ensures the uniqueness of ids across partitions.

Then we plan to start writing to the second partitions of cibuilds and cibuildsmetadata.

We will also split the existing original partition into multiple smaller partitions.

This will get us to the state where we should start to see some performance improvements. There may be some

performance decreases as we continue to read from all partitions.

We will enable the use of the partitions in self-managed for cibuilds.

Week of October 23, 2023

Team Capacity

- 1 BE

Goals

Now that we have the table name switched over in production we are moving forward with enabling the query analyzers.

We will continue working on query updates.

Task List

cibuilds

- x] Enable query analyzers for cibuilds: [Issue

Week of October 30, 2023

Team Capacity

- 1 BE

Goals

We plan to start updating the queries to include the partitionid in filtering for cibuilds and cibuildsmetadata.

Task List

cibuilds

- x] start adding partitionid filters to related queries [Issue
- ~] Switch writes to second partition: [Issue
- [~] Reintroduce migration for self-managed for ensuring uniqueness of ids across partitions

cibuildsmetadata

- ~] Switch writes to second partition: [Issue

Week of November 06, 2023

Team Capacity

- 1 BE

Goals

This week we aim to have the partitioning enabled for self-manage.

We also plan to have the required queries updated so that we can fully take advantage of partitioning for cibuilds and cibuildsmetadata.

Task List

cibuilds

- [] Complete adding partitionid filters to related queries
- [] Enable for self-managed

cibuildsmetadata

- [] Complete adding partitionid filters to related queries

</details>

Milestone 16.7 (November 13, 2023 - December 15, 2023)

Team Capacity

- 4 BE

Goals

This milestone we will finish updating queries and start writing to the second partition of cibuilds and cibuildsmetadata.

With the expanded team, we will also start the partitioning effort for cijobartifacts, cistages, and cipelinevariables.

We plan to start writing to the second partition of cibuilds and cibuildsmetadata.

We will reintroduce the migration to ensure uniqueness of ids across partitions of cibuilds and cibuildsmetadata for self-managed.

Update

We completed the critical query updates needed in order to start writing to the second partition of cibuilds and cibuildsmetadata.

We actually started writing to the second partition of cibuilds and cibuildsmetadata!

We have made significant progress on the partitioning for cipelinevariables.

We have also started work on cijobartifacts and cistages.

Milestone 16.8 (December 18, 2023 - January 12, 2024)

Team Capacity

- 4.5 BE

Goals

We plan to continue making incremental progress towards the partitioning of all 3 of cipelinevariables, cijobartifacts, and cistages tables.

This is a light milestone due to year end PTO.

Week of December 18, 2023

Team Capacity

- 4 BE

Update

We may be able to start work on cipelines as soon as this milestone. We need to swap the primary key on the table before we can start the partitioning effort. Originally we planned to wait until a backfill of foreign keys on cibuilds was complete to do the PK swap. After consulting with the database team we plan to move ahead with the swap prior to completion.

Week of December 25, 2023

Team Capacity

- 0 BE

Goals

Resting - Team members are on much deserved PTO.

Week of January 1, 2024

Team Capacity

- 0 BE

Goals

Resting - Team members are on much deserved PTO.

Week of January 8, 2024

Team Capacity

- 4 BE

Update

Given the backfill was so close to completing, we decided to wait for it to complete before moving forward with partitioning.

Milestone 16.9 (January 15, 2024 - February 9, 2024)

Team Capacity

- 5 BE

Goals

This milestone we will complete the partitioning of cipelinevariables. We will start the partitioning of cipelines.

We will continue incremental work to partition cijobartifacts, and cistages tables.

We will continue identifying and updating necessary sql queries to include partitionid.

We will flesh out work necessary for the time-decay mechanism and partition manager to dynamically create partitions.

Week of January 15, 2024

Team Capacity

- 4 BE

Update

We realized there are more indexes that need to be rebuilt for cipelines on cibuilds before we are able to swap the primary keys.

As a result, we do not think we will be able to pull in the timeline for cipelines partitioning any more than currently planned.

We also ran into a potential roadblock for re-distributing the existing large partitions into smaller ones. We may need to wait until the PG15 upgrade is complete.
Other planned work is continuing. Some team capacity is helping with regular Pipeline Execution tasks this milestone.

Week of January 22, 2024

Team Capacity

- 4 BE

Update

Week of January 29, 2024

Team Capacity

- 5 BE

Update

Task List

Week of February 5, 2024

Team Capacity

- 5 BE

Update

Milestone 16.10 (February 12, 2024 - March 8, 2024)

Team Capacity

- 6 BE

Goals

This milestone we will start work on the time decay mechanism and partition manager. Incremental work will continue towards partitioning of cijobartifacts, cistages, and cpipelines.

We will continue identifying and updating necessary sql queries to include partitionid.

Milestone 16.11 (March 11, 2024 - April 12, 2024)

Team Capacity

- 6 BE

Goals

Incremental work will continue towards partitioning of cijobartifacts, cistages, and cpipelines.

Work will continue on the tiem decay mechanism and partition manager.

We will continue identifying and updating necessary sql queries to include partitionid.

Milestone 17.0 (April 15, 2024 - May 10, 2024)

Team Capacity

- 6 BE

Goals

This milestone we will complete the partitioning of cijobartifacts and cistages.

We will continue identifying and updating necessary sql queries to include partitionid.

Milestone 17.1 (May 13, 2024 - June 14, 2024)

Team Capacity

- 6 BE

Goals

Incremental work will continue towards partitioning of cpipelines.

We will make efforts to clean up any remaining tech debt that we have created - we've been tracking it as we go.

Milestone 17.2 (June 17, 2024 - July 12, 2024)

Team Capacity

- 3 BE

Goals

This milestone we will complete the partitioning of cpipelines.

We will continue identifying and updating necessary sql queries to include partitionid.

We will make efforts to clean up any remaining tech debt that we have created - we've been tracking it as we go.

Milestone 17.3 (July 15, 2024 - Aug 9, 2024)

Team Capacity

- 3 BE

Goals

We will start using the second partition of cpipelines, cipelinevariables, cijobartifacts, and cistages.

We will start rebalancing the existing partitions now that the 6 target tables have been partitioned.

At this point it is unclear how long the rebalancing will take, but as we get closer we will update the plan with more information.

We will make efforts to clean up any remaining tech debt that we have created - we've been tracking it as we go.

Milestone 17.4 (Aug 12, 2024 - Sep 13, 2024)

Team Capacity

- 1 BE

Goals

Milestone 17.5 (Sep 16, 2024 - Oct 11, 2024)

Team Capacity

- 1 BE

Goals

Milestone 17.6 (Oct 14, 2024 - Nov 15, 2024)

Team Capacity

- 1 BE

Goals

Milestone 17.6 (Nov 18, 2024 - Dec 13, 2024)

Team Capacity

- 1 BE

Goals

We can start rebalancing old partitions using functionality in PG15 - Issue.
GitLab Postgres Upgrade Cadence

SECTION: engineering/devops/ops/verify/pipeline-execution/project-plans/ci-pk-conversion.md

CI Data Partitioning - Weekly Project Plan

Milestones Goals

- 16.9: backfill of cibuilds fk completed
- 17.0: primary key conversion to bigint for cpipelines completed

<details markdown="1">

<summary markdown="span">Archive</summary>

Week of August 7, 2023

Team Capacity

- 1 BE

Goals

Bigint conversion

- [x] Verify foreign key backfill progress for cpipelines.autocanceledbyid
- [x] Merge MR to init conversion for cisourcespipelines.pipelineid and cisourcespipelines.sourcepipelineid
- [~] Sync create index for cpipelinemessages.pipelineid (moved to 16.4 and it's under ~"workflow::in review")

Week of August 14, 2023

Team Capacity

- 1 BE

Goals

Bigint conversion

- [x] Verify foreign key backfill progress for cpipelines.autocanceledbyid, cisourcespipelines.pipelineid and cisourcespipelines.sourcepipelineid
- [~] Create foreign key constraint for cpipelinemessages.pipelineid and swap the columns (moved to 16.4)

Week of August 21, 2023

Team Capacity

- 1 BE

Goals

Bigint conversion

- [x] Verify foreign key backfill progress for cpipelines.autocanceledbyid,

cisourcespipelines.pipelineid and cisourcespipelines.sourcepipelineid

- [~] Async create index for cipipelinechatdata.pipelineid and cistages.pipelineid (under ~"workflow::in review")

Week of August 28, 2023

Team Capacity

- 1 BE

Goals

Bigint conversion

- [x] Verify foreign key backfill progress for cipipelines.autocanceledbyid
- [x] Async create index for cisourcespipelines.pipelineid and cisourcespipelines.sourcepipelineid (max 2 index creation per week)

Week of September 4, 2023

Team Capacity

- 1 BE

Goals

Bigint conversion

- [~] Async create index for cipipelines.autocanceledbyid (MR deployed, need to verify)
- [x] Sync create index for cipipelinechatdata.pipelineid
- [x] Foreign key for cipipelinechatdata.pipelineid
- [x] Sync create index for cipipelinemessages.pipelineid
- [x] Create foreign key constraint for cipipelinemessages.pipelineid and swap the columns
- [x] Async create index for cipipelinechatdata.pipelineid and cistages.pipelineid
- [~] Sync create index for cistages.pipelineid (need to wait for the async index creation)
- [~] Create foreign key constraint for cistages.pipelineid
- When postgres has upgraded to 14, init conversion for: (PG for main is still at 12.9, need to wait for the upgrade)
 - [~] pcibuilds.autocanceledbyid
 - [~] pcibuilds.upstreampipelineid
 - [~] pcibuilds.commitid

Week of September 11, 2023

Team Capacity

- 1 BE

Goals

Bigint conversion

- [~] Sync create index and foreign key constraint, and swap for cpipelines.autocanceledbyid (index)
- [~] Sync create index and foreign key constraint, and swap for cisourcespipelines.pipelineid and cisourcespipelines.sourcepipelineid
- [x] Async create index for cpipelines.autocanceledbyid (MR deployed, need to verify)
- [x] Sync create index for cistages.pipelineid (need to wait for the async index creation)
- When postgres has upgraded to 14 (this happened on Sep 12), init conversion for:
 - [~] pcibuilds.autocanceledbyid
 - [~] pcibuilds.upstreampipelineid
 - [~] pcibuilds.commitid

Milestone 16.5 (September 18, 2023 - October 16, 2023)

Week of September 18, 2023

Team Capacity

- 1 BE

Goals

Bigint conversion

Week of September 25, 2023

Team Capacity

- 1 BE

Goals

Bigint conversion

- [] Run analyze on partitioned tables
- [] Helper and documentation for swapping columns

Week of October 2, 2023

Team Capacity

- 1 BE

Goals

Bigint conversion

Week of October 9, 2023

Team Capacity

- 1 BE

Goals

Bigint conversion

Week of October 16, 2023

Team Capacity

- 1 BE

Goals

Bigint conversion

- For cpipelines.autocanceledbyid
 - [x] Sync create index (done in week 9.18)
 - [x] Async create foreign key constraint (done in week 9.25)
 - [x] Sync validate foreign key constraint
 - [~] Swap columns
- For cisourcespipelines.pipelineid and cisourcespipelines.sourcepipelineid
 - [x] Sync create index (done in week 9.25)
 - [x] Async create foreign key constraint (done in week 9.25)
 - [x] Sync validate foreign key constraint
 - [x] Swap columns
- For cipelinechatdata.pipelineid
 - [x] Swap columns
- For cipelinevariables.pipelineid
 - [~] Swap columns
- For cistages.pipelineid
 - [x] Async create foreign key constraint (done in week 9.25)
 - [x] Sync validate foreign key constraint
 - [~] Swap columns
- For cipelinemessages.pipelineid
 - [x] Sync validate foreign key constraint (done in week 9.18)
 - [x] Swap columns
- Init conversion for:
 - [~] pcibuilds.autocanceledbyid
 - [~] pcibuilds.upstreampipelineid
 - [~] pcibuilds.commitid

Milestone 16.6 (October 17, 2023 - November 10, 2023)

Team Capacity

- 1 BE

Goals

Bigint conversion

- For cpipelines.autocanceledbyid
 - [x] Swap columns
- For cipelinevariables.pipelineid
 - [x] Swap columns
- For cistages.pipelineid
 - [x] Swap columns
- Init conversion for:
 - [x] pcibuilds.autocanceledbyid
 - [x] pcibuilds.upstreampipelineid
 - [x] pcibuilds.commitid
- Stretch: Remove the triggers and integer columns for:
 - [x] cisourcespipelines.pipelineid
 - [x] cisourcespipelines.sourcepipelineid
 - [x] ciplinechatdata.pipelineid
 - [x] ciplinemessages.pipelineid
 - [~] cistages.pipelineid
 - [x] ciplinevariables.pipelineid
 - [~] cpipelines.autocanceledbyid

Milestone 16.7 (November 13, 2023 - December 8, 2023)

Team Capacity

- 1 BE

Goals

We'll monitor progress of the foreign key backfill on cibuilds this milestone.
We will continue to create the necessary MRs for updating the indexes

Bigint conversion

- Remove the triggers and integer columns for:
 - [] cistages.pipelineid
 - [] cpipelines.autocanceledbyid
- Remove the ignore rules for:
 - [x] ciplinechatdata.pipelineid
 - [] ciplinemessages.pipelineid
 - [x] cisourcespipelines.pipelineid
 - [x] cisourcespipelines.sourcepipelineid

</details>

Milestone 16.8 (December 11, 2023 - January 12, 2024)

Team Capacity

- 1 BE

Goals

We are continuing to monitor the progress of the foreign key backfill on cibuilds this milestone.

Improvements to backfill progress were made for a significant impact following the partitioning of cibuilds last milestone.

It now looks like the backfill should complete within 6 weeks rather than 6 months. As such we are pulling the planned work up.

After discussion with the database team, it seems like we are good to go ahead and swap the primary key column on cpipelines

before the backfill on cibuilds completes. As such we intend to complete that in 16.8. This will allow us to move forward with

partitioning of cpipelines while the backfile continues.

Bigint conversion

- Remove the ignore rules for:
 - [] cistages.pipelineid
 - [] cpipelines.autocanceledbyid
 - [] cpipelinevariables.pipelineid
- [] Swap columns for cpipelines.id

Milestone 16.9 (January 15, 2024 - February 9, 2024)

Team Capacity

- 1 BE

Goals

Backfill of bigint Pipeline FKs is projected to finish near the end of January 2024.

As such we should be able to create the indexes and foreign key constraints for pcibuilds.

Bigint conversion

- Create indexes and/or foreign key constraint and swap columns for:
 - [] pcibuilds.autocanceledbyid
 - [] pcibuilds.upstreampipelineid
 - [] pcibuilds.commitid

Milestone 16.10 (February 12, 2024 - March 8, 2024)

Team Capacity

- 1 BE

Goals

Backfill of bigint Pipeline FKs is projected to finish near the end of January 2024. We should be able to remove the triggers and the integer column on cpipelines once the backfill is complete and the foreign key has been updated on pcibuilds.

Bigint conversion

- [] Remove the ignore rules for cpipelines.id
- [] Remove the triggers and integer columns for cpipelines.id

Milestone 16.11 (March 11, 2024 - April 12, 2024)

Team Capacity

- 1 BE

Goals

We will plan to have the MRs for next milestone ready to go in this milestone as well given Tian's limited availability in 17.0.

Bigint conversion

- Remove the triggers and integer columns for:
 - [] pcibuilds.autocanceledbyid
 - [] pcibuilds.upstreampipelineid
 - [] pcibuilds.commitid

Milestone 17.0 (April 15, 2024 - May 10, 2024)

Team Capacity

- 0 BE (Note: Tian will be frontline responder this milestone)

Goals

Merging of the final MRs to remove the ignore rules. This should wrap up the conversion of the cpipelines PK.

There are other tables requiring PK conversion, but they are smaller and less connected. Their completion will be planned into other team work.

Bigint conversion

- Remove the ignore rules for:
 - [] pcibuilds.autocanceledbyid
 - [] pcibuilds.upstreampipelineid
 - [] pcibuilds.commitid

SECTION: engineering/devops/ops/verify/pipeline-execution/project-plans/merge-trains.md

Merge Trains - Weekly Project Plan

We have released fast-forward support for merge trains as well as some additional bug fixes while we were focused on merge train development.

We will continue to work towards maturity level "Complete" for merge trains in future milestones. For now, I am closing this particular project plan as the particular feature has shipped.

Estimated Iteration Completion Milestones

- 16.4: Initial Beta Release of Fast Forward Merge Trains and Merge Immediately without Train Restart
- 16.5: GA Release of Fast Forward Merge Trains
- 16.6: Additional Bug Fixes

<details>

<summary markdown="span">Archive</summary>

Week of August 7, 2023 (W32)

Team Capacity

- 1 Backend Engineer

Goals

- x] Merge [MR for refactoring of the Merge Request code for issue
- x] Merge [MR for creating a train ref using rebase.

Week of August 14, 2023 (W33)

Team Capacity

- 2.5 Backend Engineers

Goals

- x] [Feature flag rollout of refactoring of the Merge Request code.
- ~] Create MR(s) for [Add "Merge Immediately" option without interrupting merge train
- ~] [Feature flag rollout of creating train ref using rebase.

Week of August 21, 2023 (W34)

Team Capacity

- 2.5 Backend Engineers

Goals

- ~] Merge [MR for fast forward merge of train ref into target branch
- ~] Merge MR(s) for [Add "Merge Immediately" option without interrupting merge train
- ~] Merge MR for [Merge request stuck in locked state when getting merged in a merge train
- x] Create MR(s) for [Add "Merge Immediately" option without interrupting merge train

Week of August 28, 2023 (W35)

Team Capacity

- 2 Backend Engineers

Goals

- x] Merge [MR for fast forward merge of train ref into target branch
- x] Merge [MR for using a different commit message when doing fast forward merges
- x] Pulled forward Merge [Issue for refactoring and aggregate specs
- ~] Merge [Issue for investigating if merge commit and squashcommitsha tracking is needed
- ~] Merge MR(s) for [Add "Merge Immediately" option without interrupting merge train
- ~] Rollout feature flag for [Add "Merge Immediately" option without interrupting merge train
- x] Added Bug fix for [Issue encountered during initial rollout

Week of September 4, 2023 (W36)

Team Capacity

- 2 Backend Engineers

Goals

- ~] Merge [Issue for investigating if merge commit and squashcommitsha tracking is needed
- ~] [Feature flag rollout of creating train ref using rebase.

</details>

Week of September 11, 2023 (W37)

Team Capacity

- 2 Backend Engineers

Goals

- ~] Complete [Feature flag rollout of fast forward support or merge trains.
- ~] Merge [Issue (minor) for investigate concern that UI pipeline might not be created properly and/or status is delayed. This was raised during review of incomplete code. This might be fixed once all the parts are together)
- ~] Fast-follows for issues that arise during roll-out
- ~] Merge MR(s) for [Add "Merge Immediately" option without interrupting merge train

- x] Merge MR for [keeping track of fast-forward merge train SHAs
- x] Enable [standard commit messages for merge trains by default and on GitLab.com
- x] Enable [improved merge train ref creation by default and on GitLab.com
- x] Enable [Feature flag and one for SM of creating train ref using rebase.

Milestone 16.5 (September 18, 2023 - October 16, 2023)

Week of September 18, 2023 (W38)

Team Capacity

- 2 Backend Engineers

Goals

- x] Merge MR(s) for [Add "Merge Immediately" option without interrupting merge train
- ~] Rollout [feature flag for Add "Merge Immediately" option without interrupting merge train
- x] Complete [Feature flag rollout of fast forward support or merge trains.
- x] Merge [Issue for updating queries that associate rebased fast-forward merges with related MRs

Week of September 25, 2023 (W39)

Team Capacity

- 2 Backend Engineers

Goals

- ~] Add [developer documentation

Week of October 2, 2023 (W40)

Team Capacity

- 2 Backend Engineers

Goals

- ~] Merge MR for [Merge request stuck in locked state when getting merged in a merge train
- x] Merge [Issue (minor) for investigate concern that UI pipeline might not be created properly and/or status is delayed. This was raised during review of incomplete code. This might be fixed once all the parts are together)
- ~] Investigate and maybe merge [Issue where commit verification gets lost on fast-forward merge trains
- [x] Fast-follows for issues that arise during roll-out

Week of October 9, 2023 (W41)

Team Capacity

- 2 Backend Engineers

Goals

- ~] Complete bug fix - ["Merge" button appears instead of "Start merge train" when manual jobs are not triggered.
- ~] Duplicate - Complete bug fix - [Incorrect merge button appearing when waiting for manual action
- [x] Fast-follows for issues that arise during roll-out

Week of October 16, 2023 (W42)

Team Capacity

- 1 Backend Engineers

Goals

- [x] Any remaining fast-follows for issues that arise during roll-out
- ~] Complete bug fix - [Merge train-enabled merge request fails with "Merge request is not mergeable", succeeds on retry without changes

Milestone 16.6 (October 17, 2023 - November 10, 2023)

Team Capacity

- 1 Backend Engineer

Goals

- ~] Clean-up [Feature flag for creating train ref using rebase.
- x] Clean-up [Feature flag for fast forward support or merge trains.
- x] Clean-up feature flag for [Add "Merge Immediately" option without interrupting merge train
- ~] Merge MR(s) for [updating merge-result pipelines behind feature flag (breaking change)
- ~] Complete bug fix - [MergeTrain API failing with error Branch has been updated since the merge was requested. Please review the changes. Try again.
- ~] Complete bug fix - [Merge train-enabled merge request fails with "Merge request is not mergeable", succeeds on retry without changes

SECTION: engineering/devops/ops/verify/project-plans/_index.md

SECTION: engineering/devops/ops/verify/project-plans/ci-steps.md

Weekly Project Plan

Epic: <https://gitlab.com/groups/gitlab-org/-/epics/11535>

%16.5

Week of 2023-09-17

- Engineering strategy meeting to discuss an initial iteration plan

Week of 2023-09-25

- Clarify the iteration plan and next steps, with estimates of:
 - Architectural Design doc updates to be merged in next 2-3 weeks
 - MVC to be completed in the next 2-3 months

Week of 2023-10-02

- Iterating on technical strategy in Architectural blueprint

Week of 2023-10-09

- Iterating on technical strategy in Architectural blueprint
- Designate a Pipeline Authoring (Rails) DRI to start collaborating with the Runner DRI

%16.6

Week of 2023-10-16

- Set up sync meeting between Engineering and Product to discuss current progress and align on technical & product strategy and goals
- Iterate on the Architectural blueprint MR

Week of 2023-10-23

- Iterate on the Architectural blueprint MR
- Update the Step-Runner code based on decisions made in the blueprint

Week of 2023-10-30

- Resolved threads and merge the Architectural blueprint MR
- Update the Step-Runner code to align with the blueprint
- Implement the golang, bash and docker engines

Week of 2023-11-06

- Update the Step-Runner code to align with the blueprint
- Have it work with local Steps
- Implement the golang, bash and docker engines

%16.7

Week of 2023-11-13

- Implement the go lang, bash and docker engines
- Start determining which teams internally who could dogfood the feature

Week of 2023-11-20

- Implement the go lang, bash and docker engines

Week of 2023-11-27

- Build the Lua engine as a POC (adding in ability to script in unsupported languages)

Week of 2023-12-04

- Build the Lua engine as a POC (adding in ability to script in unsupported languages)
- Record a demo with working MVC that aligns with the blueprint

%16.8

Largely TBD given that this is a forecast 2 months out and subject to other team member availability

Week of 2023-12-11

- Build out a few more officially supported steps, such as fetch sources, caching, artifacts. (scope TBD)

Week of 2023-12-18

- Create a runner feature flag to skip built-in steps.
- Discuss feasibility of getting Runner capacity to contribute (even starting in %16.7 or %16.8 once work can be parallelized)

Week of 2023-12-25

- End of Year PTO .

Week of 2024-01-02

Discussg parallel tracks of effort (WIP plans and DRIs):

1. Implement GitLab CI Steps support in Rails. This will elevate the "steps in env variable" to a properly supported syntax, including CI Editor support. This also involves collaboration with a Rails Engineer on Pipeline Authoring.
1. Add more test coverage (unit tests, integration tests, e2e tests (and for each engine)). Up until this point, test coverage is expected to be minimal.
1. Build support for GitHub Actions in CI Steps (up until this point, only GitLab-based steps will be supported)
1. Setup publishing workflows for Step Runner and all the officially supported engines and

steps. Because users will be referencing our steps directly in their workflows, we need to publish them.

1. Write clear documentation on how users should import steps. E.g. which ones need docker, what the recommended use is, etc... I don't think the Component Catalog will be ready to curate these yet, but we can at least have some nice README's when browsing our steps sub-group.

SECTION: engineering/devops/ops/verify/runner/_index.md

Vision

By 2025, our vision for GitLab Runner is that the runner's setup and day-to-day operations at scale be an almost zero-friction experience.

Mission

Our mission is to enable organizations to efficiently run GitLab CI/CD jobs on any computing platform and do so in an operationally efficient and highly secure way at any scale.

This team maps to Verify DevOps stage.

Product Strategy and Roadmap

The product strategy and roadmap for the runner product categories are covered on the following direction pages.

- Runner Core
- Hosted Runners
- Fleet Visibility

UX strategy

Our UX vision, more information around how UX and Development collaborate, and other UX-related information will be documented in the UX Strategy page.

Our Jobs to be Done are documented in Verify:Runner JTBD and provide a high-level view of the main objectives. Our User Stories are documented in Runner Group - User Stories which guide our solutions as we create design deliverables, and ultimately map back to JTBDs.

Performance Indicator

In the OPS section, we continuously define, measure, analyze, and iterate our Performance Indicators (PIs). One of the PI process goals is to ensure that, as a product team, we are focused on strategic and operational improvements to improve leading indicators, precursors of future success.

Team Members

The following people are permanent members of the Verify:Runner group:

```
{{< team-by-manager-role "Engineering Manager(.)Verify:Runner" >}}
```

Stable Counterparts

```
{{< engineering/stable-counterparts role="Verify:Runner" manager-role="Engineering Manager(.)Verify:Runner" >}}
```

For a more comprehensive list of counterparts, look at the runner product category

Dashboards

```
{{< tableau height="600px" toolbar="hidden" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/TopEngineeringMetrics/TopEngineeringMetricsDashboard" >}}  
  {{< tableau/filters "GROUPLABEL"="runner" >}}  
{{< /tableau >}}
```

```
{{< tableau height="600px" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/MergeRequestMetrics/OverallMRsbyType1" >}}  
  {{< tableau/filters "GROUPLABEL"="runner" >}}  
{{< /tableau >}}
```

```
{{< tableau height="600px" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/Flakytestissues/FlakyTestIssues" >}}  
  {{< tableau/filters "GROUPNAME"="runner" >}}  
{{< /tableau >}}
```

```
{{< tableau height="600px" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/SlowRSpecTestsIssues/SlowRSpecTestsIssuesDashboard" >}}  
  {{< tableau/filters "GROUPLABEL"="runner" >}}  
{{< /tableau >}}
```

Projects we maintain

As a team we maintain several projects. The <https://gitlab.com/gitlab-com/runner-maintainers> group is added to each project with maintainer permission. We also try to align tools and versions used across them.

Product projects

- GitLab Runner
- GitLab Runner Operator for Kubernetes
- GitLab Runner Helm Chart
- GitLab Runner UBI offline build

Runner component projects

- Taskscaler

- Fleeting
- Fleeting Plugin AWS
- Fleeting Plugin Google Compute
- Fleeting Plugin Azure
- Fleeting Plugin Static
- Nesting
- Docker Machine (fork)
- Custom Executor Autoscaler

CI Steps projects

- Step Runner
- Action Runner

Helper projects

- Linters
 - Runner linters Docker images
 - goargs linter
- Testing
 - DinD image tests
 - SaaS Runner Tests
- Release
 - Process
 - Release tool
 - GitLab Changelog
 - Release index generator
- Maintenance
 - Runner Pod Cleanup

Runner SaaS projects

- Images
 - Linux
 - MacOS host
 - MacOS VM 12 and before
 - MacOS VM 13 and after
 - Windows
- Configuration and Deployment
 - Monitoring in K8s
 - Terraform Runner Workers (all ci and r-saas- folders)
 - Chef Runners (all runner-mananger roles)
 - Chef Runner Cookbook
 - Chef Runner Cookbook Wrapper
 - Ansible Playbooks
 - Windows Runners
 - GRIT
- Operations
 - Runbooks
 - MacOS Runbooks (deprecated)

GitLab projects that rely on Runner public-facing APIs

The following projects depend on the public Runner APIs, and should be taken into consideration in the scope of any changes/deprecations to the public API surface:

Project	API
GitLab Terraform Provider	REST API
GitLab CLI	REST API

Technologies

We spend a lot of time working in Go which is the language that GitLab Runner is written in. We also contribute to the main GitLab app, working in Rails and Vue.js. Familiarity with Docker and Kubernetes is also useful on our team.

Common Links

- Issue Board
- Issue Tracker
- Slack Channel

How we work

Iterations

We work in monthly iterations. Iteration planning dates for the upcoming milestone are aligned with GitLab's product development timeline.

At a minimum, 30 days before the start of a milestone, the runner PM reviews and re-prioritizes as needed the features to be included in the iteration planning issue. The planning issue is a tool for asynchronous collaboration between the PM, EM and members of the team. We use cross-functional prioritization to guide the collaboration process.

The commitments for the iteration plan are directly related to the capacity of the team for the upcoming iteration. Therefore, to finalize the iteration plan (resource allocation) for a milestone, we evaluate and consider the following:

- Forced prioritization issues (these issues will always be first in line for resource allocation.)
- In flight development work that did not complete prior to the feature freeze.
- Strategic direction features.
- Community or customer requested features.
- Bugs
- Technical Debt
- Maintenance
- Community merge requests review assignments

Iteration Planning and Issue Refinement Process

1. The PM creates iteration planning issues for at minimum the next three milestones.
1. The PM adds candidate issues to the planning issues, applying the appropriate priority label for the iteration (e.g. Runner::P1)
1. The PM adds the scoped label ~candidate::x.y to each issue. For example ~candidate::16.0
1. The PM assigns the iteration planning issues to the runner EM, UX, QE and TW counterparts.
1. The EM reviews all candidate tech debt, bugs, security and feature issues and applies the deliverable label to issues based on team capacity. The deliverable label signals a commitment for delivery and is tied directly to our team KPIs. Any issue not receiving the deliverable label will be treated as stretch and pulled in as team members have capacity.
1. At minimum, three business days prior to GitLab's monthly release kickoff livestream, the PM, EM, Quality and UX leads finalize the iteration plan for the upcoming milestone.

As we have a lot of involvement with our stable counterparts and reliability team, we also add a section to our iteration plan to reflect any blocking or relating issues.

1. The engineering team adds all blocking or related reliability issues to the iteration plan.
1. The reliability team reviews these issues and checks feasibility and suggests changes.
1. The reliability team commits to their issues in the iteration plan as long as:
 1. They don't affect the current due dates for an ongoing KR.
 2. They fit under one of the quarterly OKRs of the Reliability::Practices team.
 3. They take into account downtime related to the OnCall and OnCall follow up work.

Prioritization labeling

To indicate priority of issues during an iteration we may use labels ~"Runner::P1" ~"Runner::P2" ~"Runner::P3".

At a minimum we will always identify our top priorities using ~"Runner::P1".

1. ~"Runner::P1" means "elevated priority". We aim to deliver all or most of these issues.
1. ~"Runner::P2" means "normal priority".
1. ~"Runner::P3" means "reduced priority".
1. ~"Runner::P" labels can and should differ from ~priority: labels.
 - ~priority: labels imply the timeline for when issues will be addressed.
 - While ~"Runner::P" indicate priority for the scheduled iteration.

Design and development process

We follow the product development flow. Our team uses one issue as SSOT for design, backend, and frontend work.

Once a problem is validated, the issue enters the design phase where the product designer collaborates with the team to ideate solutions and explore different approaches before converging on a single solution that is feasible and has requirements meet the business goals.

Sometimes we need to increase our confidence that the proposed solution meets the user's needs and expectations. This confidence can be obtained from additional research during the solution validation phase.

Following the design and validation phases, the problem should already be broken down into the quickest change possible to improve the user's outcome and be ready for a more detailed review

by engineering before moving to the build track.

Once the PM intends to prioritize the issue for the next milestone, the ~"workflow::planning breakdown" label is applied and the EM will assign a developer to further break down and apply weights to that work so that the issue can be ~"workflow::ready for development".

Release

At the end of the iteration we release Runner and associated projects. The release process is documented [here](#).

Guidelines for Merge Requests

As a developer on the runner team, you will be contributing to the various runner projects. Since the GitLab Runner project reviewers and maintainers review all code contributions (runner team members and community contributions), we must try and be as efficient as possible when submitting merge requests for review.

The responsibility of the merge request author

We follow the merge request author responsibility guidelines.

The responsibility of Reviewers and Maintainers

We follow the code review guidelines.

To help authors find a reviewer with capacity to take on a review, we have a spreadsheet dashboard that shows the number of MRs any of the backend members of the Verify:Runner or Verify:Runner SaaS groups have assigned.

If you as a reviewer or maintainer who has reached your limit of assigned review MRs, consider asking for assistance from your peers by reassigning some to them. Additionally consider pair-reviewing with the authors on a video call to speed up the review cycle - especially if you have multiple MRs to review from a single author.

Non-team member MRs count towards WIP limit. At GitLab anyone can contribute, and codebases do not equal "teams" or "groups" (even if they happen to share a name). Therefore we should, from time to time, anticipate the occasional MR from a non-team member. Since other teams may not be familiar with our imposed WIP limits, we will need to accommodate them as best we can and the reviewers may need to help with the re-balancing their workload. We should not accept these MRs as a valid reason to go above the WIP limits.

These limits are intended to help with the work load on the reviewers and maintainers. If you are feeling pressured to rush through reviews, talk to your EM. Quality is always more important than speed of review.

Runner Group Specific Onboarding Needs

- editor access to the group-verify project in GCP
- Add as maintainer to the gitlab-com/runner-group group on GitLab.com

- Make sure entry in team.yml has the new member as a reviewer of gitlab-org/gitlab-runner and gitlab-org/ci-cd/custom-executor-drivers/autoscaler
- Add to Verify 1password vault (requires creating an access request).

Onboarding

When a new developer joins Runner, their responsibility will include maintaining the runner project and all satellite repositories we own from their first day. This means that the developer will get Maintainer access to our repositories and will be added to the runner-maintainers group so they appear in merge request approval group.

This allows the onboarding developer to grow organically over time in their responsibilities, which might include (non-exhaustive) code reviews, incident response, operations and releases. We should still follow the traditional two-stage review process for merges in most cases (incident response and operations being exceptions if the situation warrants it).

Becoming a maintainer for one of our projects

Although maintainer access is provided from day one for practical purposes, we follow the same process outlined here.

Any engineer inside of the organization is welcome to become a maintainer of a project owned by the Runner team.

Technical Debt / Backstage work

In general, technical debt, backstage work, or other classifications of development work that don't directly contribute to a users experience with the runner are handled the same way as features or bugs and covered by the above Kanban style process. The one exception is that for each engineer on the team, they can only have 1 technical debt issue in flight at a time. This means that if they start working on a technical debt type issue they cannot start another one until the first one is merged. In the event that an engineer has more than one technical debt item in flight, they should choose which one to keep working on and move the others to the "in development" or "ready for review" columns depending on their status. The intent of this limitation is to constrain the number of technical debt issues that are in review at any given time to help ensure we always have most of our capacity available to review and iterate on features or bugs.

Retrospectives

The team has a monthly retrospective meeting on the first Tuesday of the month. The agenda can be found [here](#) (internal link).

Deprecations process

At GitLab, our release post policy specifies that deprecation notices need to be added to the release post at least two cycles before the release when the feature is removed or officially obsolete. There are typically several deprecations or removals that the runner team needs to manage across the main runner project and the other projects that this team maintains. As

such, the runner development team uses the following process to manage deprecations and removals. This process should start no later than one month after the launch of a major release.

1. The assigned developer creates a Deprecations and Removal epic for the next major release. See example epic.

1. The assigned developer collects all planned deprecations and removals with input from the development team and includes them in the epic.

1. The assigned developer verifies that there are deprecation issues created for each deprecation.

1. The assigned developer tags the runner development team, engineering manager, and product manager.

1. The product manager uses the list of issues to create the deprecation notices. Our goal is to start announcing deprecations no later than six cycles before the next major release.

1. The product manager will continue to include the deprecation notices in all release post entries up to and including the major release where the features will be fully deprecated or removed.

Managing CVE vulnerability report issues

Managing CVE vulnerability issues is part of GitLab's vulnerability management effort (1, 2), and is an important part of maintaining the GitLab FedRAMP certification.

Using the container-scanners project, GitLab scans all images we produce to highlight CVE vulnerabilities. From those scans, the vulnmapper

project creates issues in the project that created the vulnerable image, including SLAs to which we must adhere.

The Runner team member assigned the Support & Security Responder role in the weekly team task should triage and review the list of CVEs and address any issues as appropriate:

- Critical severity issues should be addressed immediately.
- High, Medium, and Low severity issues should be addressed in the priority order of the remediation SLAs.

The procedure for addressing CVE issues is as follows:

Surfacing active vulnerability reports

- Use one of the following to surface active CVE issues assigned to our team:
 - Issue search
 - The gitlab-dashboard cves command.
 - The cver imageVulns command.

Many issues will reference the same CVE vulnerability report; it's best to group issues for the same vulnerability report and address them together.

- Focusing on CVE reports in priority order, start with critical, high, and medium severities first and proceed as follows:

1. For each group of common/related issues, confirm that the associated CVE is still valid.

This can be done by

scanning the latest version of the image(s) identified in the issue(s) with tools such as trivy and gype, and checking whether the CVE referenced in the issue appears in the trivy or gype scan.

1. If the vulnerability is no longer reported in the trivy or gype scan of the relevant image(s), the issue(s) can be closed. Note that the cver internal tool mentioned above largely automates this task, including closing the relevant issues (see the documentation).

1. If the vulnerability is still present in the relevant image(s), it must be addressed.

Note that issues that reference ubi-fips flavors of gitlab-runner or gitlab-runner-helper images take precedence over other image flavors (like alpine or ubuntu) since the GitLab FedRAMP certification is contingent on ubi-fips images only.

Addressing active vulnerability reports

Vulnerabilities usually appear in one of three flavors (ordered in most to least frequency of occurrence):

- The vulnerability exists in a third-party OS package (like git or git-lfs).
- The vulnerability exists in gitlab-runner in one of its dependencies.
- The vulnerability exists in gitlab-runner in code we've written.

Third-party OS packages

In this case, the vulnerability:

- Has not been fixed upstream
- Has been fixed upstream but an OS package including the fix has not been created and published yet
- Will not be fixed upstream

The primary course of action here is to create a deviation request issue

(see

<https://handbook.gitlab.com/handbook/security/security-assurance/security-compliance/poam-deviation-request-procedure/>).

We generally create one deviation request issue per offending software module (e.g. git-lfs or libcurl). When

creating the issue, be sure to select operationalrequirementtemplate as a template and complete the following

sections:

- Affected images
- Vulnerability details (one row for each relevant CVE report)
- Relevant vulnmapper issues
- Justification

Once the deviation request issue is created, add:

- A note to all the relevant gitlab-runner issues pointing to the deviation request issue
- The label FedRAMP::DR Status::Open
- The most relevant label from this list:
 - Vulnerability::Vendor Base Container::Fix Unavailable
 - Vulnerability::Vendor Base Container::Will Not Be Fixed
 - Vulnerability::Vendor Package::Fix Unavailable
 - Vulnerability::Vendor Package::Will Not Be Fixed

Eventually, a fix in the offending package will make its way to the OS package manager, and then both the gitlab-runner and deviation request issues can be closed.

gitlab-runner dependencies

The simplest course of action here is to update the dependency to the latest compatible version (or at least a version that addresses the vulnerability). Once the MR with the dependency update is merged, the gitlab-runner issue can be closed.

If the dependency does not address the vulnerability, possible courses of action are:

- If a fork of the dependency that addresses the vulnerability exists, use it with the Go module replace directive. In this case, be sure to create a task to switch back to the upstream dependency when the vulnerability has been addressed there.
- If possible, consider not using the dependency or replacing it with another similar dependency.
- Create a deviation request issue.

gitlab-runner source

The only course of action here is to fix the vulnerable code. If the fix is not simple and will take time to implement (and prevent us from meeting CVE SLAs), it might be necessary to create a deviation request issue.

Working with security forks

When issues are marked confidential, the MR that fixes the issue should be made in a project's security fork (see

security-forks). In general the process is identical to creating and merging MRs in the canonical project repo, with a couple of notable differences.

Note that MRs in the security repo must be reviewed/approved by a security counterpart in addition to a runner code-owner.

The examples below are given for the GitLab Runner project, but apply equally to all runner-related projects with security forks.

Keeping the security fork up to date with its canonical repo

Security forks are configured to automatically synchronize with the canonical repo, but this can be disabled if changes exist in the security fork's main branch that do not exist in the canonical repo's main branch. This usually happens when a security MR is merged into the security fork's main, but not into the canonical repo's main branch. In this event, it is necessary to manually synchronize the security fork against the canonical repo.

From a checked-out canonical repo:

shell

`git fetch` ensure you have the latest changes from the canonical repo.

`git remote add security git@gitlab.com:gitlab-org/security/gitlab-runner.git` add the security repo as a remote, be sure to use the git url.

`git fetch security` fetch the security fork repo references.

`git checkout -b security-main security/main` checkout the security fork's main branch.

`git rebase --rebase-merges origin/main` rebase the canonical main onto the security main.

`git log --color --topo-order --oneline` ensure the resulting history is sane.

`git push --force` push the resulting local security main branch to the security remote repo.

Notes:

1. These steps will not fully synchronize the security and canonical repositories in both directions. They will only bring changes that are only in the canonical repo, into the security repo. Synchronizing in the other direction is described below.
2. The security repos do/should not have force-push branch protection on the main branch, but if the one you are working with does, temporarily disable it so you can perform the last step.
3. If the security fork main branch becomes too out of date with the canonical repo main branch (specifically with changes that exist only in the security repo), merge conflicts are likely